[GOVERNMENT COLLEGE UNIVERSITY HYD]
[INFORMATION TECHNOLOG Department]

CardioCheck

Heart Size Estimation from Chest X-rays
Using the Cardiothoracic Ratio

Project Report

SUBMITTED BY

Name	Ali Asghar, Nouma Shaikh
Roll number	BSIT-2024-074 & BSIT-2024-072
Supervisor	MR. Amjad Ali Khawaja
Program	Information Technology
Course	Artificial Intelligence
Semester	6th
Section	A

Contents

1. Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope 1
2. Background 2
2.1 Cardiothoracic Ratio 2
2.2 Image Segmentation and U-Net 2
2.3 Pretrained Model 2
3. Tools and System Design 3
3.1 Technologies Used 3
3.2 Architecture 3
3.3 Workflow 3
3.4 Project Structure 4
3.5 API 4
3.6 Classification Levels 5
4. Implementation 6
4.1 Analysis Module (model.py) 6
4.2 Server (server.py) 8
4.3 Frontend 8
4.4 Installation and Running 9
5. Results 10
5.1 User Interface 10
5.2 Sample With a Normal Result 11
5.3 Sample With Suspected Cardiomegaly 12
5.4 Overlay Controls 13
5.5 Discussion 13
6. Limitations and Future Work 14
6.1 Limitations 14
6.2 Privacy and Security 14
6.3 Future Work 14
7. Conclusion 15
References 16
Appendix A: Frontend Script (script.js) 17

1. Introduction

1.1 Background

A chest X-ray is one of the most common imaging tests in medicine. It is quick, cheap and available in almost every hospital. Doctors use it to look at the lungs, the ribs and the outline of the heart. One of the simplest things that can be read from a front-view film is the size of the heart. An enlarged heart, called cardiomegaly, can be a sign of several heart conditions, so it is worth noticing early.

The usual way to judge heart size on a plain film is the cardiothoracic ratio (CTR). It compares the widest part of the heart with the widest inner part of the chest. In adults, a ratio above 0.50 on a standard front-view film is generally taken as a sign that the heart may be enlarged. Done by hand, the measurement needs calipers or a ruler on the film and some judgement about where the edges are, so two people can end up with slightly different numbers.

1.2 Problem Statement

Measuring the CTR manually is slow, depends on the person doing it, and is hard to repeat the same way every time. We wanted to see how much of this routine step can be automated with a pretrained deep learning model, and to present the result in a form that a person without a technical background can read at once.

1.3 Objectives

The aims of this project were the following.

- To segment the heart and both lungs from a front-view chest X-ray using a pretrained U-Net.
- To compute the cardiothoracic ratio from the segmentation masks and classify the result.
- To draw the measurements on the image so the user can check what the program actually measured.
- To build a simple web interface where an X-ray can be uploaded and analysed in a few seconds on an ordinary computer, without a GPU.

1.4 Scope

CardioCheck is a student project made for learning and demonstration. It works on front-view (PA or AP) chest X-rays in PNG or JPEG format and runs entirely on a local machine. It is not a

medical device. It has not been validated for clinical use and must not be used to diagnose or make decisions about anyone's health.

2. Background

2.1 Cardiothoracic Ratio

The CTR is the maximum horizontal width of the heart divided by the maximum horizontal width of the chest. In the standard method the chest width is taken at the inner edge of the ribs. The calculation is

$$CTR = \text{heart width} / \text{chest width}$$

A value of 0.50 or below is considered normal for an adult on a front-view (PA) film. Children normally have a higher ratio, and films taken from the front to back (AP), often with the patient lying down, make the heart look larger than it is. For this reason the application shows the detected view and warns the user when the image is an AP film.

2.2 Image Segmentation and U-Net

Segmentation means labelling every pixel of an image with the class it belongs to. In a chest X-ray the classes of interest are the right lung, the left lung and the heart. U-Net is a convolutional network designed for this kind of task. It has an encoder that reduces the image to a compact set of features and a decoder that expands those features back to full size. Connections between matching encoder and decoder levels keep fine spatial detail, which is important for getting clean outlines of organs.

2.3 Pretrained Model

Training a segmentation network from scratch needs a large set of labelled X-rays and a lot of computing time, which was not realistic for this project. We therefore used a model that is already trained and published on Hugging Face, called ianpan/chest-x-ray-basic. Its main properties are listed in Table 1.

Table 1. Properties of the pretrained model

Property	Description
Architecture	U-Net with an EfficientNetV2-S encoder
Size	About 22.2 million parameters, roughly 90 MB
Outputs	Masks for the right lung, left lung and heart, and a view class (AP, PA or Lateral)
Training data	Chest X-rays from CheXpert and the NIH Chest X-ray dataset, with masks from CheXmask
Published view accuracy	99.42 percent, as reported by the model author

The accuracy figure comes from the model's own documentation. We did not run our own evaluation, so it describes the model as published and not this application.

3. Tools and System Design

3.1 Technologies Used

The project uses Python on the server side and plain HTML, CSS and JavaScript in the browser. Table 2 lists the main tools.

Table 2. Tools and libraries

Layer	Tools
Backend	Python 3, Flask
Model	PyTorch, Hugging Face transformers, ianpan/chest-x-ray-basic
Image handling	OpenCV, NumPy
Frontend	HTML, Tailwind CSS (compiled into style.css), vanilla JavaScript
Fonts	Inter, JetBrains Mono and Material Symbols, served locally

3.2 Architecture

The application has three parts. The browser page lets the user choose or upload an image. A small Flask server receives the image and hands it to the analysis module. The analysis module runs the pretrained model, measures the widths and builds the result. The model is loaded once when the server starts and is reused for every request. Nothing is written to disk, so an uploaded image exists only in memory while it is being processed. Figure 1 shows how the parts are connected.

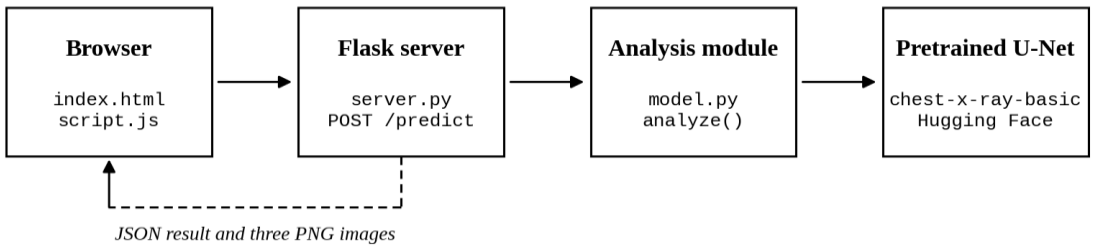


Figure 1. Overall structure of CardioCheck

3.3 Workflow

A single analysis goes through the following steps.

1. The user uploads a PNG or JPEG chest X-ray, or picks one of the two built-in samples.
2. The browser sends the file to the server with a POST request to /predict.
3. model.py decodes the image as grayscale and preprocesses it for the model.

4. The model predicts the view and the masks. Lateral images are rejected with a clear message.
5. From the masks, the widest horizontal extent of the heart and of both lungs is measured in pixels.
6. The CTR is calculated as heart width divided by lung width and rounded to two decimals.
7. The server returns the numbers together with four PNG images: the annotated picture, the plain X-ray, the heart overlay and the measurement lines.
8. The page draws the overlays over the plain X-ray, so each one can be switched on and off.

3.4 Project Structure

```
.
├── server.py         Flask app: serves the page and exposes POST /predict
├── model.py          Loads the model and runs the analysis
├── README.md
├── LICENSE
├── docs/
│   └── screenshots/   Images used in the README
└── public/
    ├── index.html     Page markup
    ├── style.css      Compiled Tailwind styles
    ├── script.js      Upload, API calls, overlays, result display
    ├── fonts/         Local font files
    ├── images/        Favicons and the hero image
    └── samples/       Sample chest X-rays
```

3.5 API

The server exposes one endpoint, POST /predict. The image is sent as multipart/form-data in a field named image. On success the server returns JSON with the fields in Table 3.

Table 3. Fields in the response of /predict

Field	Meaning
view	AP or PA
heart_width	Horizontal extent of the heart mask, in pixels
chest_width	Horizontal extent of both lung masks, in pixels
ctr	heart_width / chest_width, to two decimals
result	"Possible cardiomegaly" if the CTR is above 0.50, otherwise "Normal heart size"
image	Full picture with the heart mask and both lines drawn in
base	Plain X-ray with no overlay

Field	Meaning
heart_layer	Transparent PNG with the heart mask
lines_layer	Transparent PNG with the two measurement lines

Errors are returned as JSON in the form {"error": "message"}. Status 400 is used when no image is sent, the file cannot be decoded, the film is a lateral view, or the heart or lungs are not found. Status 413 is returned when the file is larger than 10 MB. Widths are in pixels of the model's preprocessed image, not in millimetres.

3.6 Classification Levels

The interface groups the CTR into three levels, shown in Table 4. The /predict endpoint itself uses a single cut-off of 0.50 for its result string, and the three-level grouping is done in script.js. The two are consistent for the normal case, but the Borderline level exists only in the interface.

Table 4. CTR levels used in the interface

CTR	Level
0.50 or below	Normal
0.51 to 0.55	Borderline
Above 0.55	Suspected cardiomegaly

4. Implementation

4.1 Analysis Module (model.py)

This file does the actual work. The model is loaded once at import time with AutoModel.from_pretrained, and the function analyze takes the raw bytes of an image and returns a dictionary of results. The code first decodes the image as grayscale and runs the model without gradient tracking. If the predicted view is lateral, or if no heart or lung pixels are found, it raises a ValueError with a message that the server passes on to the user.

The masks use the labels 1 for the right lung, 2 for the left lung and 3 for the heart. The widths are taken from the smallest and largest column that contains each mask. The lung width stands in for the inner rib width of the clinical method, so the result approximates the real CTR but is not identical to it. The function then builds two transparent layers, one for the heart and one for the two measurement lines, as well as a combined picture, and encodes them as base64 PNG strings that the browser can show directly. The complete file is given in Listing 1.

Listing 1. model.py

```
import base64
import cv2
import numpy as np
import torch
from transformers import AutoModel

# load the model once, when this file is first imported
model = AutoModel.from_pretrained("ianpan/chest-x-ray-basic", trust_remote_code=True)
model.eval()



def _png(arr):
    """numpy image -> PNG data URL the browser can show"""
    _, buf = cv2.imencode(".png", arr)
    return "" + base64.b64encode(buf).decode()



def analyze(image_bytes):
    """Takes the raw bytes of an X-ray image and returns the results as a dictionary.
    Raises ValueError with a friendly message if the image can't be used."""

    data = np.frombuffer(image_bytes, np.uint8)
    img = cv2.imdecode(data, cv2.IMREAD_GRAYSCALE)
    if img is None:
        raise ValueError("Could not read this image")

    x = model.preprocess(img)
    x = torch.from_numpy(x).unsqueeze(0).unsqueeze(0).float()

    with torch.inference_mode():
        out = model(x)

    view = ["AP", "PA", "Lateral"][out["view"].argmax(1).item()]
```

```
if view == "Lateral":
    raise ValueError("This is a side-view X-ray. Please upload a front-view chest X-ray.")

# 1 = right lung, 2 = left lung, 3 = heart
mask = out["mask"].argmax(1)[0].numpy()
heart = mask == 3
lungs = (mask == 1) | (mask == 2)

if not heart.any() or not lungs.any():
    raise ValueError("Heart or lungs not found. Use a front-view chest X-ray.")

# Horizontal spans of the heart and lung masks. Lung width stands in for the
# inner rib width, so this approximates the clinical cardiothoracic ratio.
_, lx = np.where(lungs)
_, hx = np.where(heart)

lung_width = int(lx.max() - lx.min())
heart_width = int(hx.max() - hx.min())

if lung_width <= 0:
    raise ValueError("Could not measure the lung segmentation.")

ctr = round(heart_width / lung_width, 2)
result = "Possible cardiomegaly" if ctr > 0.5 else "Normal heart size"

# plain X-ray (no overlay)
base = x[0, 0].numpy()
base = (base - base.min()) / (base.max() - base.min() + 1e-8)
base = (base * 255).astype(np.uint8)
h, w = base.shape

row = int(np.where(heart)[0].mean())
heart_line = ((int(hx.min()), row), (int(hx.max()), row))
chest_line = ((int(lx.min()), row + 12), (int(lx.max()), row + 12))

# two transparent layers, so the page can show or hide each one
heart_layer = np.zeros((h, w, 4), np.uint8)
heart_layer[heart] = (0, 0, 255, 128)  # BGRA: red, half transparent
lines_layer = np.zeros((h, w, 4), np.uint8)
cv2.line(lines_layer, *heart_line, (0, 255, 255, 255), 2)  # yellow
cv2.line(lines_layer, *chest_line, (255, 255, 0, 255), 2)  # cyan

# full picture with everything drawn in (used by the hero image)
pic = cv2.cvtColor(base, cv2.COLOR_GRAY2BGR)
red = pic.copy()
red[heart] = (0, 0, 255)
pic = cv2.addWeighted(red, 0.5, pic, 0.5, 0)
cv2.line(pic, *heart_line, (0, 255, 255), 2)
cv2.line(pic, *chest_line, (255, 255, 0), 2)

return {
    "view": view,
    "heart_width": heart_width,
    "chest_width": lung_width,
    "ctr": round(ctr, 2),
    "result": result,
    "image": _png(pic),
    "base": _png(base),
    "heart_layer": _png(heart_layer),
    "lines_layer": _png(lines_layer),
}
```

4.2 Server (server.py)

The server is a short Flask application. It serves the files in the public folder, accepts the upload on /predict, passes the bytes to model.analyze and returns the result as JSON. Errors raised by the analysis module become a JSON message with status 400. The upload size is limited to 10 MB with MAX_CONTENT_LENGTH. The server runs with debug=False. Listing 2 shows the file.

Listing 2. server.py

```
from flask import Flask, request, jsonify, send_from_directory
import model  # our AI file: model.py

# serve everything in the "public" folder
app = Flask(__name__, static_folder="public", static_url_path="")
app.config["MAX_CONTENT_LENGTH"] = 10 * 1024 * 1024  # 10 MB limit



@app.route("/")
def home():
    return send_from_directory("public", "index.html")



@app.route("/predict", methods=["POST"])
def predict():
    if "image" not in request.files:
        return jsonify(error="No image uploaded"), 400

    try:
        result = model.analyze(request.files["image"].read())
    except ValueError as e:
        return jsonify(error=str(e)), 400

    return jsonify(result)



if __name__ == "__main__":
    app.run(debug=False)
```

4.3 Frontend

The interface is a single page, index.html, styled with Tailwind CSS that has been compiled into style.css. The page has a short introduction, an interactive section where the analysis is done, and explanations of how the method works, what the model is, what the CTR means and what the limitations are. All fonts are served from the project folder, so the page works without an internet connection after the model has been downloaded once.

The behaviour of the page is in script.js, which is listed in full in Appendix A. It handles the file upload and drag and drop, sends the image to the server, and fills in the result. When the page loads it analyses the first sample automatically and then the second sample in the background, so both sample badges show a real value. Each overlay is a separate transparent image placed

over the plain X-ray, which is why the buttons can hide the heart outline and the measurement lines without another request to the server. The page also shows a warning when the image is an AP view.

4.4 Installation and Running

Python 3.9 or newer is needed, with about 1 GB of free disk space for PyTorch and the model weights. An internet connection is required on the first run to download the model from Hugging Face. The steps are shown below. The first start is slow because the weights are downloaded and cached; later starts load them from the cache.

```
python -m venv .venv
.venv\Scripts\activate        (Windows)
source .venv/bin/activate      (macOS / Linux)

pip install flask opencv-python-headless numpy torch transformers
python server.py
```

The application is then available at http://127.0.0.1:5000 in the browser.

5. Results

5.1 User Interface

Figure 2 shows the complete page. The top of the page introduces the tool, followed by the interactive demo and the explanation sections.

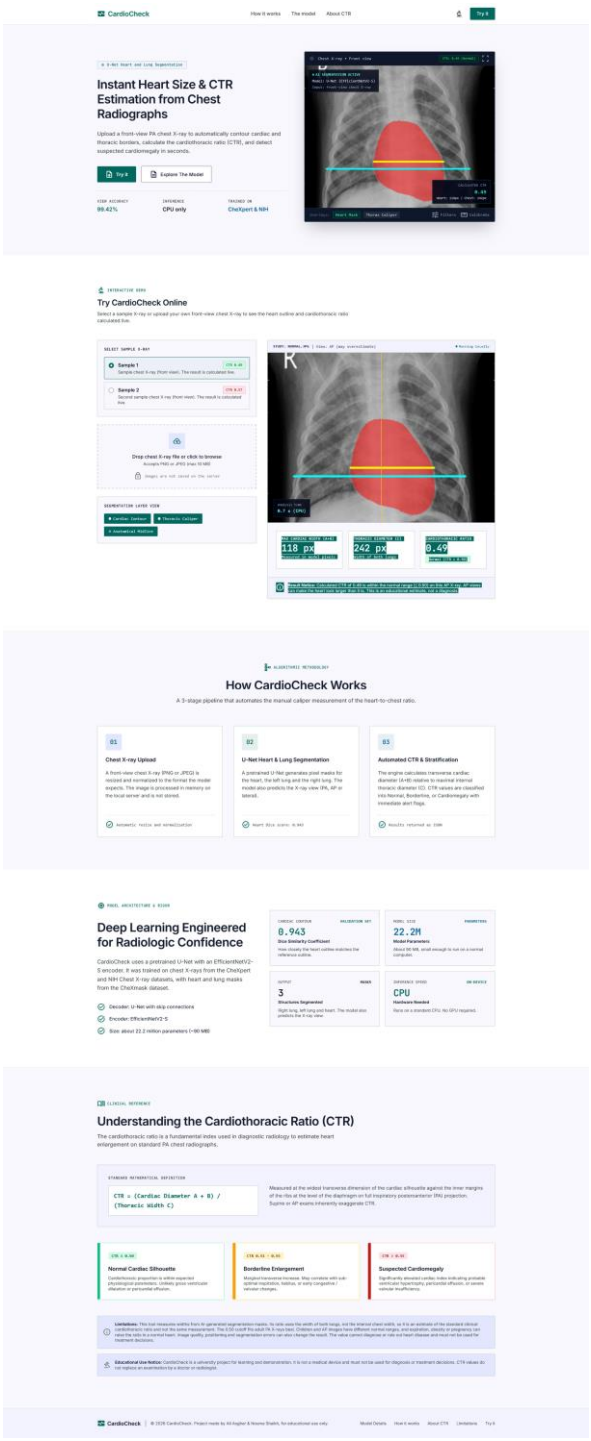


Figure 2. Complete CardioCheck page

5.2 Sample With a Normal Result

Figure 3 shows the first sample. The heart is marked in red, the yellow line is the heart width and the cyan line is the chest width. The values read from the screen are listed in Table 5.

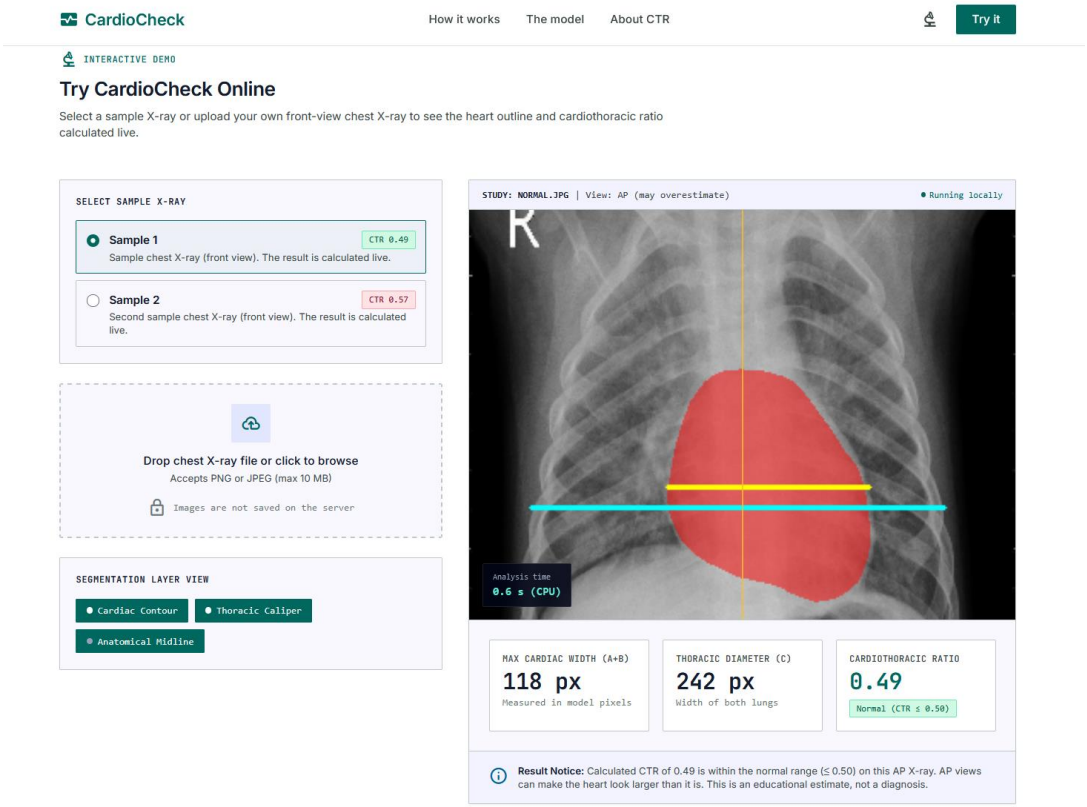


Figure 3. Result for the first sample image

Table 5. Measurements for the first sample

Item	Value
Detected view	AP
Heart width	118 px
Chest width	242 px
CTR	0.49
Status	Normal (CTR 0.50 or below)
Analysis time	0.6 s on CPU

5.3 Sample With Suspected Cardiomegaly

The second sample gives a higher ratio. As shown in Figure 4 and Table 6, the CTR is above 0.55, so the interface shows a red status badge.

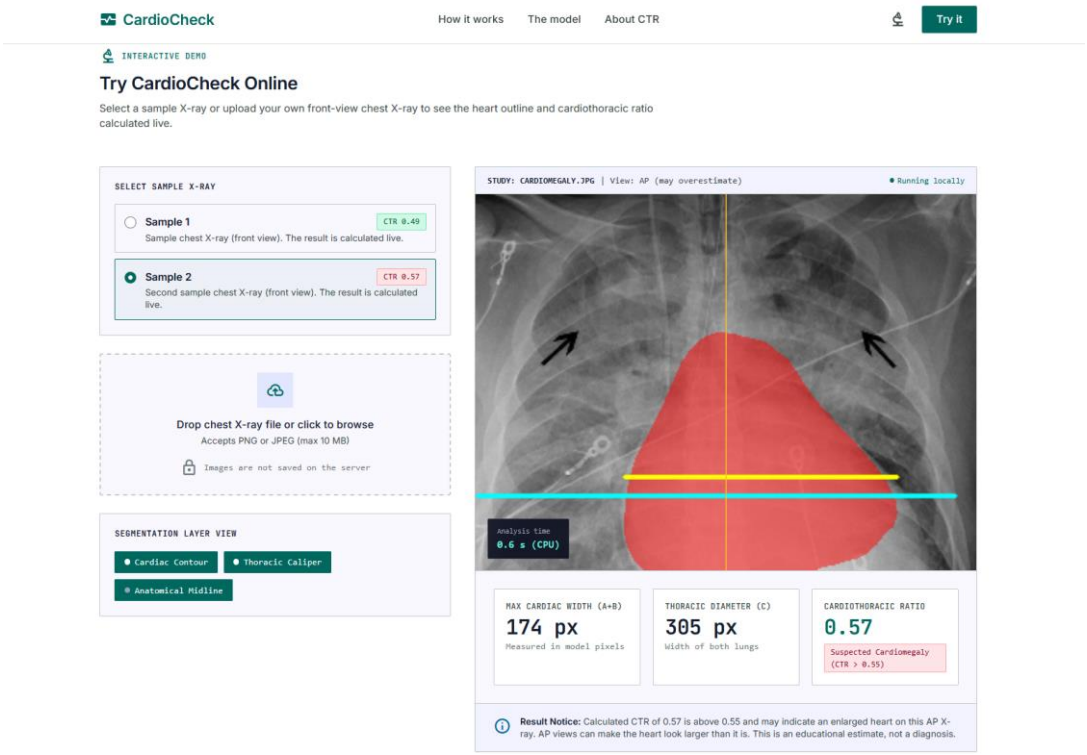


Figure 4. Result for the second sample image

Table 6. Measurements for the second sample

Item	Value
Detected view	AP
Heart width	174 px
Chest width	305 px
CTR	0.57
Status	Suspected cardiomegaly (CTR above 0.55)
Analysis time	0.6 s on CPU

5.4 Overlay Controls

The layer buttons hide the heart outline and the measurement lines, which makes it possible to look at the plain X-ray and to check whether the outline really follows the heart. Figure 5 shows the viewer with both overlays switched off.

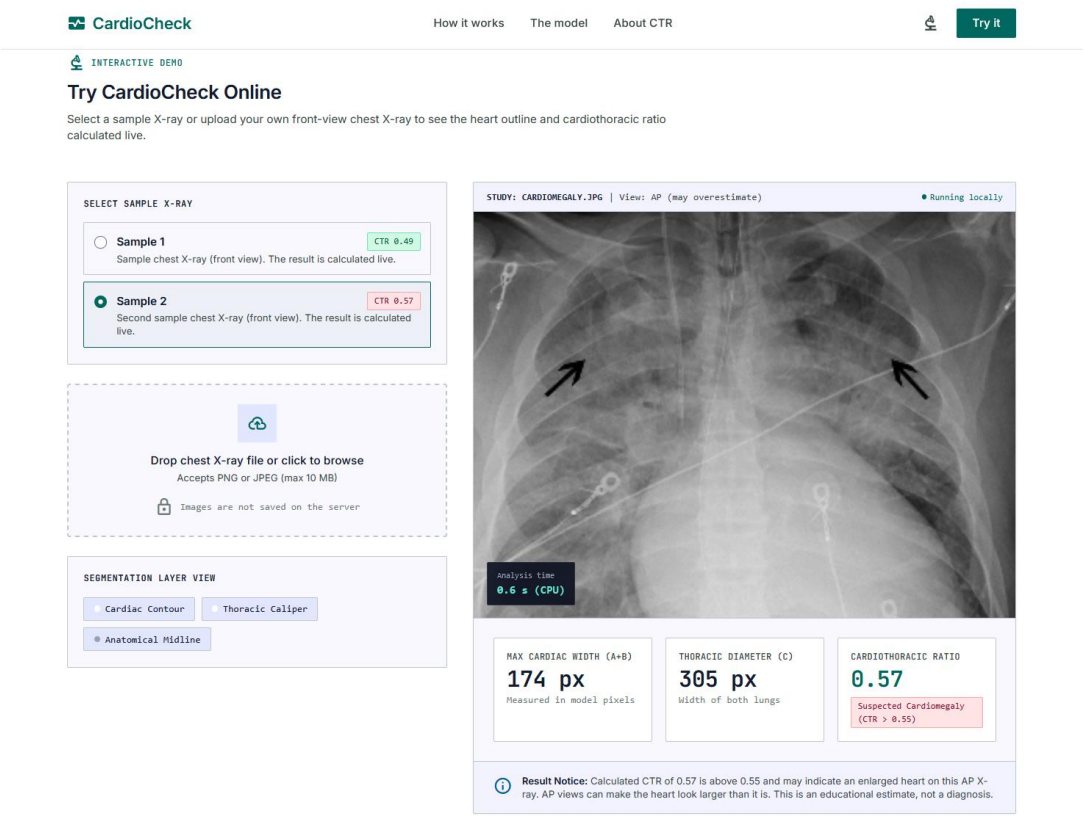


Figure 5. Viewer with the overlays switched off

5.5 Discussion

On the two built-in samples the program separates the cases in the expected way: a ratio of 0.49 for the first image and 0.57 for the second. Each analysis took well under a second on a normal laptop CPU. Both images were detected as AP views, so the page also displayed the warning that AP films can make the heart look larger. Two images are not enough to judge accuracy. We did not measure the CTR on a labelled set or compare it with measurements made by a radiologist, so we make no claim about how close the values are to a manual reading.

6. Limitations and Future Work

6.1 Limitations

- A CTR from a single image is a rough estimate, and two radiologists can disagree on the same film.
- The standard method measures the chest width at the inner rib margin. This project uses the width of both lung masks, which usually gives a slightly different value.
- The 0.50 cut-off is meant for adult PA films. Children have a higher normal range, and AP films overestimate the heart size.
- Poor inspiration, rotation, pacemakers, large pleural effusions or unusual anatomy can cause bad segmentation. The code only checks that a heart and lungs were found, not that the outlines are correct.
- A normal CTR does not rule out heart disease, and a high CTR does not confirm it.
- The model was not tested or cleared for clinical use, and its published accuracy was not verified by us.

6.2 Privacy and Security

Uploaded images are processed in memory and are not saved. Images with patient identifiers should not be used. The model is loaded with `trust_remote_code=True`, which runs Python code from the Hugging Face repository on the local machine, so a specific revision should be pinned once that code has been reviewed. The built-in Flask server is suitable for a demonstration on one computer. A public deployment would need a production server, HTTPS and rate limiting.

6.3 Future Work

- Evaluate the CTR against radiologist measurements on a labelled set of images.
- Measure the chest width at the inner rib margin instead of using the lung masks.
- Use age-appropriate thresholds for children.
- Check the quality of the segmentation and warn the user when an outline looks wrong.
- Add a pinned `requirements.txt` and a production-ready deployment.

7. Conclusion

CardioCheck turns the manual cardiothoracic ratio measurement into a short automatic step. A pretrained U-Net segments the heart and lungs, the program measures their widths and computes the ratio, and a web page shows the numbers together with the outlines that were used, so the result can be checked by eye. The project runs on a normal computer without a GPU and gives an answer in about a second.

The main limit of the work is that it has not been tested against reference measurements, so it should be seen as a working demonstration of the idea and not as a diagnostic tool. Even so, building it gave us practical experience with using a pretrained model, handling images in Python, designing a small API and connecting it to a browser interface.

References

[1] Pan, I. chest-x-ray-basic [pretrained model]. Hugging Face. https://huggingface.co/ianpan/chest-x-ray-basic

[2] Ronneberger, O., Fischer, P., and Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. Medical Image Computing and Computer-Assisted Intervention (MICCAI), pp. 234–241.

[3] Tan, M., and Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training. Proceedings of the 38th International Conference on Machine Learning (ICML).

[4] Irvin, J., et al. (2019). CheXpert: A large chest radiograph dataset with uncertainty labels and expert comparison. Proceedings of the AAAI Conference on Artificial Intelligence.

[5] Wang, X., et al. (2017). ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

[6] Gaggion, N., et al. (2024). CheXmask: A large-scale dataset of anatomical segmentation masks for multi-source chest X-ray images. Scientific Data.

[7] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems (NeurIPS).

[8] Flask documentation. Pallets Projects. https://flask.palletsprojects.com

[9] CardioCheck source code. https://github.com/aliasghar-codes/Cardio-Check

Appendix A: Frontend Script (script.js)

This file controls the page: file selection, calls to the server, overlay switching and the display of results.

Listing 3. public/script.js

```
const $ = (id) => document.getElementById(id);
const input = $("radiograph-upload");
const SAMPLES = { normal: "samples/normal.jpg", cardiomegaly: "samples/cardiomegaly.jpg" };

// three CTR levels, same as the "About CTR" cards
const TONE = {
  ok: { short: "Normal", label: "Normal (CTR ≤ 0.50)", badge: "bg-emerald-100 text-emerald-800 border-emerald-300", hero: "bg-
      emerald-950/80 border-emerald-500/40 text-emerald-400", num: "text-emerald-400", note: "is within the normal range (≤
      0.50)" },
  border: { short: "Borderline", label: "Borderline (CTR 0.51 – 0.55)", badge: "bg-amber-100 text-amber-900 border-amber-300",
      hero: "bg-amber-950/80 border-amber-500/40 text-amber-400", num: "text-amber-400", note: "is borderline (0.51 – 0.55)" },
  high: { short: "Suspected", label: "Suspected Cardiomegaly (CTR > 0.55)", badge: "bg-rose-100 text-rose-900 border-rose-300",
      hero: "bg-rose-950/80 border-rose-500/40 text-rose-400", num: "text-rose-400", note: "is above 0.55 and may indicate an
      enlarged heart" },
};
const level = (ctr) => (ctr <= 0.5 ? TONE.ok : ctr <= 0.55 ? TONE.border : TONE.high);

const BADGE = "inline-flex items-center px-2 py-0.5 rounded text-[11px] font-mono border font-medium ";
const MINI = "px-2 py-0.5 rounded text-[11px] font-mono border ";
const HERO_BADGE = "inline-flex items-center px-2 py-0.5 rounded text-[10px] font-mono border ";
const HERO_NUM = "font-bold text-base ";
const CARD = "flex items-start gap-3 p-3 rounded border cursor-pointer transition-all ";
const CARD_ON = CARD + "border-primary bg-primary/5";
const CARD_OFF = CARD + "border-outline-variant hover:border-outline";

let busy = false;

const setNotice = (text) => ($("notice-text").textContent = text);

function setBusy(on) {
  busy = on;
  input.disabled = on;
  document.querySelectorAll('input[name="sample_selection"]').forEach((r) => (r.disabled = on));
  document.body.classList.toggle("cursor-wait", on);
  $("notice-text").classList.toggle("animate-pulse", on);
  $("preview-xray").classList.toggle("opacity-50", on);
}

// each layer is its own transparent picture on top of the plain X-ray
function setLayer(id, src) {
  const img = new Image();
  img.src = src;
  img.alt = "";
  img.className = "w-full h-full object-cover";
  $(id).replaceChildren(img);
}

function toggleLayer(name) {
  const hidden = $(`layer-${name}`).classList.toggle("hidden");
  const on = !hidden;
  const btn = $(`toggle-${name}-mask`);
  ["bg-primary", "text-on-primary", "border-primary"].forEach((c) => btn.classList.toggle(c, on));
  ["bg-surface-container-high", "text-on-surface", "border-outline-variant"].forEach((c) => btn.classList.toggle(c, !on));
}
```

```
async function predict(blob, name) {
  const form = new FormData();
  form.append("image", blob, name);
  const t0 = performance.now();
  const res = await fetch("/predict", { method: "POST", body: form });
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(data.error || "Upload failed. The file may be too large (max 10 MB).");
  data.seconds = (performance.now() - t0) / 1000;
  return data;
}

function showResult(data, blob, name) {
  if (!data.base || !data.heart_layer || !data.lines_layer) {
    return setNotice("The server is still running the old heart_model.py. Replace it with the new one and restart python app.py.");
  }
  const t = level(data.ctr);
  $("preview-xray").src = data.base;
  setLayer("layer-heart", data.heart_layer);
  setLayer("layer-thorax", data.lines_layer);

  $("val-cardiac").textContent = `${data.heart_width} px`;
  $("val-thorax").textContent = `${data.chest_width} px`;
  $("val-ctr").textContent = data.ctr.toFixed(2);
  $("badge-ctr-status").className = BADGE + t.badge;
  $("badge-ctr-status").textContent = t.label;
  $("active-study-id").textContent = "STUDY: " + name.toUpperCase();
  $("study-meta").textContent = "| View: " + data.view + (data.view === "AP" ? " (may overestimate)" : "");
  $("latency").textContent = `${data.seconds.toFixed(1)} s (CPU)`;

  setNotice(
    `Calculated CTR of ${data.ctr.toFixed(2)} ${t.note} on this ${data.view} X-ray.` +
    (data.view === "AP" ? " AP views can make the heart look larger than it is." : "") +
    " This is an educational estimate, not a diagnosis."
  );
}

function fillSampleBadge(id, data) {
  $(id).className = MINI + level(data.ctr).badge;
  $(id).textContent = `CTR ${data.ctr.toFixed(2)}`;
}

function fillHero(data) {
  const t = level(data.ctr);
  $("hero-xray").src = data.image;
  $("hero-badge").className = HERO_BADGE + t.hero;
  $("hero-badge").textContent = `CTR: ${data.ctr.toFixed(2)} (${t.short})`;
  $("hero-ctr").className = HERO_NUM + t.num;
  $("hero-ctr").textContent = data.ctr.toFixed(2);
  $("hero-widths").textContent = `Heart: ${data.heart_width}px / Chest: ${data.chest_width}px`;
}

async function loadSample(type) {
  const url = SAMPLES[type];
  const res = await fetch(url);
  if (!res.ok) throw new Error(`Sample image not found: public/${url}`);
  return { blob: await res.blob(), name: url.split("/").pop() };
}

async function run(getInput, after) {
  if (busy) return;
  setBusy(true);
  setNotice("Analyzing X-ray...");
```

```
  try {
    const { blob, name } = await getInput();
    const data = await predict(blob, name);
    showResult(data, blob, name);
    if (after) after(data);
  } catch (e) {
    setNotice(e instanceof TypeError ? "Could not reach the server. Is it running?" : e.message);
  } finally {
    setBusy(false);
  }
}

function switchSample(type) {
  if (busy) return;
  $("sample-btn-1").className = type === "normal" ? CARD_ON : CARD_OFF;
  $("sample-btn-2").className = type === "normal" ? CARD_OFF : CARD_ON;
  const badge = type === "normal" ? "sample1-ctr" : "sample2-ctr";
  run(() => loadSample(type), (d) => fillSampleBadge(badge, d));
}

function pick(file) {
  if (busy) return;
  document.querySelectorAll('input[name="sample_selection"]').forEach((r) => (r.checked = false));
  $("sample-btn-1").className = CARD_OFF;
  $("sample-btn-2").className = CARD_OFF;
  run(async () => ({ blob: file, name: file.name }));
}

input.addEventListener("change", () => {
  if (input.files[0]) pick(input.files[0]);
  input.value = "";
});

const dropzone = document.querySelector('label[for="radiograph-upload"]').parentElement;
dropzone.addEventListener("dragover", (e) => e.preventDefault());
dropzone.addEventListener("drop", (e) => {
  e.preventDefault();
  if (e.dataTransfer.files[0]) pick(e.dataTransfer.files[0]);
});

// on page load: analyze sample 1 (fills the result, hero and its badge),
// then sample 2 in the background so its badge shows a real value too
async function init() {
  await run(() => loadSample("normal"), (d) => {
    fillSampleBadge("sample1-ctr", d);
    fillHero(d);
  });
  try {
    const { blob, name } = await loadSample("cardiomegaly");
    fillSampleBadge("sample2-ctr", await predict(blob, name));
  } catch (e) { /* sample 2 missing: leave its badge as "CTR --" */ }
}
init();
```